Self-Improving RAG: System Architecture & Technical Report

A Production & Research-Grade Retrieval-Augmented Generation Engine with Active Learning

System Status: Milestones 1–4 Fully Operational & Verified | **Cost Footprint:** $0 (Local Open-Source Compute + Free Tier Groq API)
Primary Technologies: FastAPI, SQLite (WAL Mode), FAISS, Okapi BM25, Cross-Encoder, SentenceTransformers, Groq (qwen/qwen3.8-27b)

---

1. Executive Summary & Problem Formulation

Conventional Retrieval-Augmented Generation (RAG) pipelines suffer from critical engineering shortcomings: **(1) Dense retrieval blind spots** on lexical queries, part numbers, and domain acronyms; **(2) Silent hallucinations** and ungrounded fabrications that simplistic word-overlap heuristics fail to detect; **(3) Static prompt drift** where LLM generation repeatedly makes the same mistakes despite human feedback; and **(4) Unreliable JSON flat-file storage** causing file corruption and write conflicts under concurrency.

This project resolves these failure modes through an autonomous, **Self-Improving RAG** architecture. The system incorporates automated evaluation via the **RAG Triad** (Faithfulness, Answer Relevancy, Context Precision), a state-of-the-art **Hybrid Retrieval Engine** (Okapi BM25 + FAISS Dense Embeddings fused via Reciprocal Rank Fusion and re-ranked by a Cross-Encoder), an **Active Learning Episodic Memory** that dynamically injects corrective guidelines at inference time, and an **ACID-compliant SQLite relational persistence layer** operating in Write-Ahead Logging (WAL) mode. Crucially, the entire architecture runs on a **$0 cost budget** utilizing local open-source models and Groq's high-speed inference.

Key Architectural Capabilities:
- **Triad Evaluation:** Dual-engine LLM-as-a-judge with deterministic SentenceTransformer semantic fallback.
- **Hybrid Retrieval:** Okapi BM25 sparse index + FAISS dense search + RRF fusion + ms-marco Cross-Encoder re-ranker.
- **Active Self-Correction:** In-context dynamic prompt adaptation + semantic failure clustering (DBSCAN).
- **ACID Persistence:** SQLite WAL-mode transactional store replacing flat JSON files.

2. End-to-End Query Lifecycle & Subsystems

Phase	Module & Models	Input / Output	Operational Description
1. Ingestion	ingest.py all-MiniLM-L6-v2, FAISS, BM25 Index	Raw Text / Markdown → 384d Vectors, Inverted Index	Documents are recursively split, embedded into dense vectors, and synced into FAISS IndexFlatIP and Okapi BM25 vocabulary.
2. Hybrid Retrieval	retrieve.py, bm25.py BM25, FAISS, RRF, Cross-Encoder	User Query → Re-ranked Context Chunks	Expands candidate pool based on failure memory, merges dense & sparse ranks via RRF (k=60), and re-ranks via joint cross-attention.
3. Adaptive Generation	generate.py, exemplars.py Groq (qwen/qwen3.8-27b)	Query + Contexts + Exemplars → Grounded Answer	Retrieves past user corrections for similar queries. Dynamically adapts the system prompt with episodic guidelines before LLM inference.
4. Triad Audit & DB	evaluate.py, db.py LLM Judge / Fallback, SQLite	Answer + Context + Ground Truth → Triad Scores + Traces	Computes Faithfulness, Relevancy, and Precision. If Faithfulness < 0.60, rejects as hallucination. Logs all traces to SQLite (WAL).

3. Detailed Engineering Milestones & Methodologies

3.1 Milestone 1: RAG Triad Evaluation Engine (Faithfulness, Relevancy, Precision)

Standard word-overlap heuristics (like Jaccard or ROUGE-1) fail to detect hallucinations and context relevance. We implemented the standardized **RAG Triad framework** (aligned with Ragas research standards):

• **Faithfulness (Hallucination Detection):** Evaluates whether every atomic claim in the generated answer is directly grounded in the retrieved context. If claims extrapolate or invent facts, Faithfulness drops to 0.0, immediately triggering a rejection flag.

• **Answer Relevancy:** Assesses how directly and completely the answer addresses the user's explicit query, penalizing conversational filler and off-topic tangents.

• **Context Precision:** Evaluates signal-to-noise ratio in retrieved context chunks, rewarding rankings where relevant information appears at rank 1.

• **Dual-Engine Reliability:** Uses primary *LLM-as-a-judge* with structured JSON outputs. If Groq hits rate limits (1000 OTPM on free tier) or errors, the evaluator seamlessly falls back to local *SentenceTransformer semantic embedding similarity*.

3.2 Milestone 2: Hybrid Retrieval & Cross-Encoder Re-Ranking

Dense embeddings struggle with rare tokens, exact IDs, and domain-specific keywords. The hybrid retrieval engine bridges this gap:

• **Okapi BM25 Index (app/bm25.py):** Zero-dependency sparse inverted index with term saturation (*k1=1.5*) and document length penalization (*b=0.75*).

• **Reciprocal Rank Fusion (RRF):** Combines dense vector rankings (FAISS cosine similarity) and sparse lexical rankings (BM25 scores) using rank consensus: RRF(d) = Σ [1 / (60 + rank_m(d))]. This prevents one scoring space from distorting another.

• **Cross-Encoder Re-Ranking:** The top candidate chunks from RRF are re-scored using cross-encoder/ms-marco-MiniLM-L-6-v2. Unlike bi-encoders which process query and document independently, cross-encoders execute full token-to-token attention across the pair, achieving state-of-the-art precision for Top-K extraction.

• **Multi-Mode API:** The retrieval endpoint dynamically supports hybrid (default), dense, and sparse modes.

3.3 Milestone 3: Active Feedback Loop & In-Context Episodic Learning

Rather than treating human feedback as passive telemetry, the self-improving engine closes the active learning loop:

• **Episodic Memory Store (app/exemplars.py):** When bad feedback contains a human correction or guideline, it is embedded and indexed into episodic memory.

• **Dynamic Prompt Adaptation (app/generate.py):** At inference time, incoming queries are compared against past failure exemplars. If a semantic match occurs, the system dynamically injects corrective guidelines directly into the LLM system prompt.

• **Adaptive Retrieval Policy:** Queries matching recurring failure topics automatically expand their candidate retrieval pool (e.g. 2× top_k).

• **Offline Semantic Failure Clustering:** The POST /feedback/optimize endpoint groups flagged queries using DBSCAN/cosine similarity, revealing recurring blind spots in the knowledge base.

3.4 Milestone 4: Relational SQLite Storage & WAL Concurrency

Replaced fragile, race-condition-prone flat JSON logging (feedback_log.json, failed_queries.json) with an ACID-compliant relational SQLite store (db/rag_store.db):

• **Write-Ahead Logging (WAL Mode):** Enabled via PRAGMA journal_mode=WAL, allowing simultaneous readers without blocking writers.

• **Relational Schema:** Fully indexed tables for queries (latency & trace tracking), evaluations (triad metrics), feedback, failed_queries (failure frequency counter), and exemplars (guidelines and vectors).

• **Automatic Zero-Loss Migration:** On application launch, init_db() automatically detects legacy JSON files and migrates all historical records into the SQLite tables with conflict resolution.

4. Comprehensive REST API Reference

The system exposes a unified, production-ready FastAPI interface with strict Pydantic schemas:

Method & Endpoint	Key Parameters / Payload	Response & Functionality
`GET /`	None	System health check returning active LLM provider, embedding model, and device status.
`GET /db/status`	None	SQLite health check, WAL journal mode confirmation, database file size, and table row counts.
`POST /ingest`	texts: list[str]	Splits documents, computes embeddings, updates FAISS index and BM25 vocabulary.
`POST /retrieve`	query: str, mode: 'hybrid'|'dense'|'sparse', top_k: int, rerank: bool	Retrieves ranked contexts using BM25, FAISS, RRF fusion, and cross-encoder re-ranking.
`POST /generate`	query: str, context_docs: list[str], max_tokens: int, temperature: float	Generates grounded answer; dynamically injects retrieved past corrective exemplars.
`POST /evaluate`	query: str, answer: str, context_docs: list[str], ground_truth: str	Scores Faithfulness, Answer Relevancy, Context Precision; flags hallucinations.
`POST /ask`	query: str, top_k: int, mode: str, evaluate: bool	**Full Pipeline:** Retrieves, adapts prompt, generates answer, runs RAG Triad audit, logs query trace to DB.
`POST /feedback`	query: str, score: 'good'|'bad', correction: str, failure_reason: str	Logs rating to DB. If bad with correction, registers new episodic exemplar.
`GET /feedback/exemplars`	limit: int = 10	Returns all active learned episodic corrections stored in SQLite memory.
`POST /feedback/optimize`	None	Executes DBSCAN semantic clustering on failed queries; outputs cluster insights.
`GET /feedback/stats`	None	Returns aggregated feedback metrics (total, good, bad %, avg score).
`GET /feedback/improvements`	None	Returns recurring failure topics, cluster summaries, and actionable knowledge base fixes.

5. Empirical Test Suite & Verification Results

The automated test harness (run_tests.py) validates all integrated components across 13 exhaustive test stages. All test runs succeeded with zero errors:

Test Stage	Scope / Focus Area	Observed Metric / Output	Status
TEST 1 & 1b	Health Check & SQLite WAL Health	Engine: SQLite | Journal: WAL | Size: 76KB | 5 tables initialized	**PASSED**
TEST 2a	Hybrid Search (BM25 + FAISS + Cross-Encoder)	Candidate pool fused via RRF; top score = 1.0; 2 docs reranked	**PASSED**
TEST 2b & 2c	Dense-only vs. Sparse-only Retrieval	Dense retrieves neural attention doc; Sparse retrieves exact FAISS keyword doc	**PASSED**
TEST 3	Full RAG Pipeline (POST /ask)	Latency: 148ms retrieval, 1610ms generation; Composite Score = 1.0	**PASSED**
TEST 4a & 4b	RAG Triad (Standalone & Combined)	Faithfulness: 1.0 | Answer Relevancy: 1.0 | Context Precision: 1.0	**PASSED**
TEST 4c	Hallucination / Ungrounded Claim Detection	Faithfulness: 0.0 < 0.60 threshold → Flagged as hallucinated and rejected	**PASSED**
TEST 5	Positive Feedback Recording	Logged to SQLite feedback table with unique UUID	**PASSED**
TEST 6a & 6b	Corrective Exemplar Learning	Bad feedback + user correction registered into SQLite episodic memory	**PASSED**
TEST 6c	Dynamic Runtime Prompt Adaptation	Query on semantically similar topic retrieved exemplar and adapted prompt	**PASSED**
TEST 6d	Semantic Failure Topic Clustering	Clustered 7 historical failures into 1 cluster with actionable fix hints	**PASSED**
TEST 7 & 8	Feedback Analytics & Actionable Hints	Real-time aggregation: 15 feedback entries, 53.3% good, prioritized query list	**PASSED**

6. Project Repository Architecture & Conclusion

```
selfimprovingrag/
■■■ app/ (ask.py, bm25.py, db.py, evaluate.py, exemplars.py, feedback.py, generate.py, ingest.py, main.py, retrieve.py)
■■■ db/ (faiss.index, rag_store.db [SQLite WAL])
■■■ config.py (thresholds, weights, models)  |  run_tests.py (13 E2E test stages)  |  test_api.py (TestClient suite)
```

Conclusion: The Self-Improving RAG system delivers a research-grade and production-ready architecture. By uniting sparse lexical search (BM25) and dense embeddings (FAISS) with cross-encoder re-ranking, auditing every answer against the RAG Triad, adapting generation dynamically from human corrections, and guaranteeing ACID persistence via SQLite WAL, the system establishes a zero-cost blueprint for autonomous, continuously improving retrieval-augmented applications.